Towards Efficient & Adaptable Hardware for Multicore Workloads

Keshav Krishna,
Supervisor: Professor Mohamed Zahran

Abstract

Modern multicore workloads shift across threads, processes, and resource bottlenecks, making static hardware policies inefficient for power, thermal, and performance control. This paper studies a compact resource-family phase interface for efficient and adaptable multicore hardware. Offline train-only clustering over safe hardware counters defines interpretable low-, moderate-, and high-pressure states; the online predictor then approximates those labels using clustered histories of one validation-selected counter from each resource family. Across three PARSEC-derived regimes, a shallow cross-family decision tree reaches 69.6%, 76.0%, and 91.5% mean held-out accuracy with only 156–307 B of model state per family predictor. It remains within 0.8 percentage points of, matches, or exceeds a tiny Transformer upper bound while using 227–447$\times$ less storage. These results suggest that offline phase discovery can be distilled into low-cost hardware-facing signals organized by resource family, making them directly useful for future power-, thermal-, cache-, and scheduler-control policies.

1 Introduction

Modern multicore systems are asked to sustain increasingly large workloads: multithreaded applications, colocated services, background analytics, and mixed software stacks that compete for shared caches, memory bandwidth, power budget, and thermal headroom. These workloads are not steady. They move between compute-heavy, memory-intensive, branch-sensitive, synchronization-heavy, and interference-dominated intervals as thread count, process mix, and input phase change. When hardware treats that changing behavior as static, it either provisions for the worst case and wastes power, or it underprovisions active phases and loses performance. The result is a familiar systems tension: high throughput requires aggressive hardware resources, while energy-efficient operation requires those resources to be activated only when they are useful.

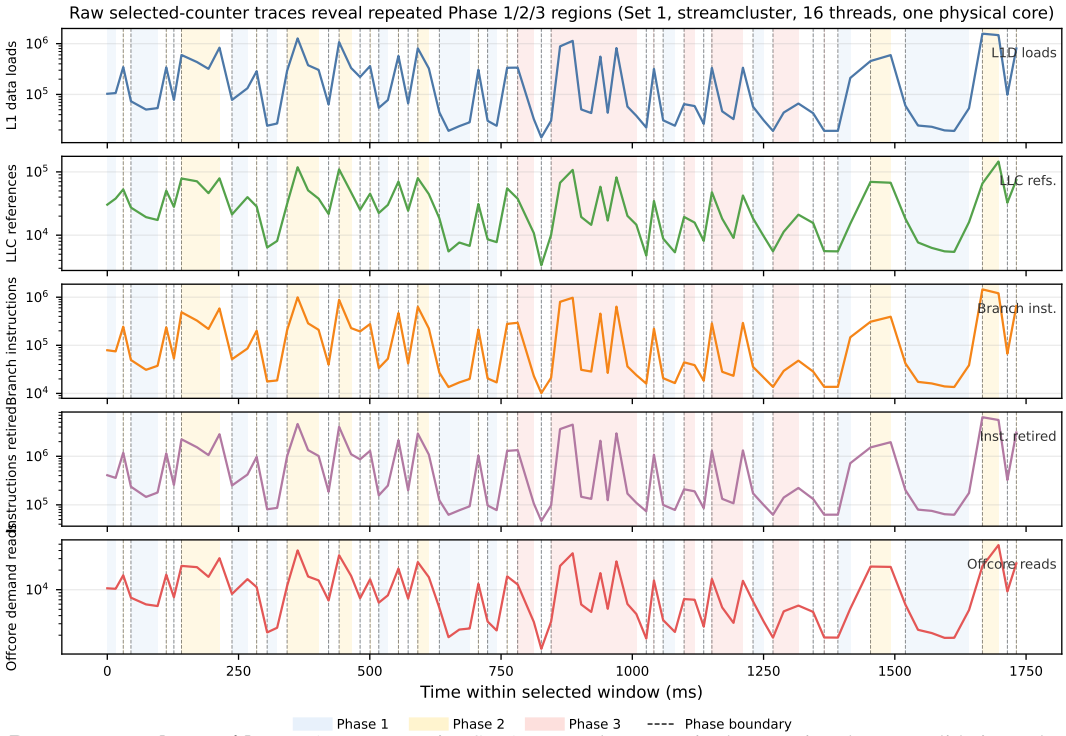
Adaptable hardware is the natural response. Dynamic voltage/frequency scaling can lower power when work is memory-bound or lightly parallel, cache and prefetch policies can change with locality and bandwidth pressure, power gating can idle unused structures, and schedulers can place threads according to interference and resource demand. These controls matter because power is no longer just a secondary metric. In dense multicore systems, wasted activity increases energy cost, reduces thermal margin, and can force throttling that harms throughput. However, adaptation only helps if the hardware can recognize behavioral changes quickly and cheaply. A controller that depends on a large software model, many simultaneous performance counters, or online clustering may consume too much state, latency, and complexity for the tight loops where power and resource decisions are made.

Program phases provide the missing abstraction. A phase representation is more stable than raw counter fluctuations but more responsive than a static workload label. This broad control problem appears in cache reconfiguration, adaptive queues, runtime power monitoring, power balancing, and heterogeneous scheduling [3, 1, 4, 18, 27, 8, 30, 5]. Prior work has shown that phases can be exploited for simulation-point selection, runtime adaptation, prediction, and multiprocessor management [31, 32, 13, 33, 6, 29, 11, 20, 36, 28, 25, 5].

The remaining challenge, and the central obstacle to adaptable hardware, is deployment cost. Offline BBV and SimPoint-style methods are powerful, but they are profiling-oriented rather than direct hardware implementations. Runtime clustering requires centroid maintenance and can be sensitive to short-lived fluctuations. Neural sequence models can improve prediction, but their storage, arithmetic cost, and software stack requirements make them awkward inside short control loops. Meanwhile, practical PMU designs expose only a small number of counters at once, and many counters are redundant. The semantics and availability of those counters are also architecture- and toolchain-dependent, which makes event selection itself a systems problem rather than a purely statistical one [12, 14, 16, 17, 22, 23].

This paper asks a deliberately constrained hardware question: how much multicore phase-prediction fidelity can be recovered after aggressively shrinking the online interface and organizing it around hardware resource families rather than cores? The proposed flow uses train-only clustering over the full safe counter vector to define offline phase labels, selects one counter per resource family by validation-set ablation, converts each selected counter into a three-state train-only clustered resource-family signal, and then predicts the next phase from short histories of those clustered states. This reframes the problem from per-core phase ownership or threshold bucketing to teacher-student distillation over resource families: the expensive clustering teacher exists only offline, while the deployed model sees a reduced, discrete interface that could feed low-latency power, cache, prefetch, or scheduling decisions.

The completed final run produces three main results for the efficient-and-adaptable hardware narrative. First, the ordered cluster labels are interpretable as low-, moderate-, and high-pressure states rather than arbitrary IDs. Second, cross-family



Phase 1: Raw-counter phase evidence. A representative Set 1 streamcluster run is shown using the raw validation-selected counters for five resource families. The shaded regions are the train-only clustered Phase 1, Phase 2, and Phase 3 labels, and dashed lines mark the resulting phase boundaries. The repeated structure is visible directly in the counter traces, motivating the use of resource-family phases as compact hardware-facing control signals.

clustered-state trees recover most of the accuracy of much larger software baselines with two to three orders of magnitude less storage. Third, the accuracy/transition trade-off depends strongly on concurrency regime: persistence baselines remain hard to beat in raw accuracy when phase runs are long, but history-aware models recover non-zero transition sensitivity and become more useful as behavior becomes less stable.

This paper makes the following contributions toward hardware-efficient multicore adaptation:

- A counter-independence analysis that uses correlation structure to identify redundant hardware-counter indicators before model training, reducing the monitored event set without relying on hand-picked thresholds.
- An ablation workflow that evaluates which candidate counters best capture resource-family phase behavior.
- A per-family representative-counter study for L1, LLC, branch/control, core/FP, and memory/off-core behavior, showing which existing PMU events best preserve resource-family phase information across the studied multicore regimes.
- A phase-analysis comparison across single-process multithreaded, multiprocess interference, and hybrid multiprocess/multithread regimes, showing how phase persistence, transition rate, and detector accuracy change with workload construction.
- An online resource-family phase-detection and next-phase prediction analysis in which deployed models consume clustered histories of the selected counters with accuracy, transition behavior, latency, and hardware-budget estimates.
- A hardware-oriented interpretation of resource-family phase definitions and how these phase signals can support adaptable control policies for power, thermal, cache, prefetch, and scheduling decisions.

2 Background and Hardware Design Goals

2.1 What Is a Phase?

A program phase is a set of execution intervals that behave similarly according to some chosen representation. This definition is intentionally broader than a single contiguous time region. In SimPoint-style analysis, two intervals may belong to the same phase even if they occur far apart in time, as long as their basic-block vectors or other fingerprints indicate similar behavior [31, 32, 13]. Runtime phase trackers use the same principle online: they compare compact signatures or hardware-counter vectors against previously observed behavior and assign a phase label when the current interval is close enough to a

known pattern [33, 28].

The practical meaning of a phase therefore depends on the signal used to define it. Code-centric definitions, such as basic-block vectors, are hardware independent and good for simulation-point selection, but they require profiling or code signatures. Working-set definitions capture memory footprint and are useful for reconfigurable caches and other multi-configuration hardware, but they may miss branch or compute behavior [9, 10]. Hardware-counter definitions are easier to observe at runtime and naturally expose cache, branch, memory, and instruction behavior, but their meaning depends on available PMU events, multiplexing, and architecture-specific event semantics [12, 16, 17, 22]. Clustered definitions, including the one used in this paper, treat phase construction as an unsupervised grouping problem over a selected feature space; they can combine several resource dimensions but must be handled carefully to avoid leakage and unstable labels [26, 24, 19, 25].

2.2 Phase Definitions in Multicore Systems

Multicore execution adds a second question: whose behavior does the phase describe? Prior multicore studies distinguish global, local, and distributed phase definitions, each with different implications for accuracy, overhead, and hardware control [29, 6, 25]. Table 1 summarizes the main trade-offs.

The global definition is attractive for hardware because it minimizes state, but it can be too coarse for modern multicore workloads. Perelman et al. show that parallel phase behavior depends on thread scaling and shared-memory interaction, while Alcorta Lozano and Gerstlauer report that per-core definitions are often stronger for workload forecasting even when global labels minimize within-phase variation [29, 25]. Purely local phases, however, do not explain whether a core is slow because of its own behavior or because another core is pressuring the LLC or memory system. Distributed definitions address this by feeding cross-core information into the phase model, but the added context can also inject noise and raise implementation cost [25]. This paper takes a different axis through the design space. Instead of defining phases primarily by core, thread, or whole-processor ownership, it defines the online phase interface by hardware resource family: L1, LLC, branch/control, core instruction/FP, and memory/off-core behavior. That resource-family view is less about identifying which core changed and more about identifying which hardware resource class is entering a low-, moderate-, or high-pressure state. The distinction matters for adaptation because a cache phase can naturally drive cache or prefetch policy, a branch/control phase can inform front-end behavior, and a memory/off-core phase can guide bandwidth, scheduling, or frequency decisions.

2.3 Phase-Dependent Hardware Resources

Different phase definitions are useful only if they connect to resources that hardware can control or observe cheaply. The literature points to three recurring resource classes. First, cache and memory-hierarchy demand is strongly phase dependent: Ziedan et al. estimate per-phase L2 cache demand directly, Perelman et al. validate parallel phase quality using L2/L3 hit-rate stability, and Alcorta Lozano and Gerstlauer include L2 misses and off-core demand reads to capture memory boundedness [36, 29, 25]. Second, control-flow behavior matters because branch mispredictions waste pipeline work independently of cache capacity; branch counts and branch-miss counters appear in phase and power/performance predictors, including P^4 and the multicore forecasting study [20, 25]. Third, frequency and execution-resource allocation are natural phase-dependent controls: compute-heavy phases may benefit from higher frequency or wider execution resources, while memory-bound phases may save energy at lower frequency because the core is often waiting on data [5, 20, 28].

This motivates using counters that are already available on common PMUs rather than proposing new sensors. L1/L2/LLC activity, branch behavior, retired instructions, off-core reads, and memory-bandwidth proxies are not identical across vendors, but they represent resource classes that most modern processors expose in some form [16, 17, 22, 23]. The correlation and ablation stages in this paper follow that principle: they avoid tracking several highly correlated indicators and keep the online interface to one representative event per resource family.

These observations motivate three design goals for a predictor intended to support efficient and adaptable multicore hardware. First, the phase labels should reflect combined workload behavior rather than manual per-counter buckets. Second, the online representation should be small enough to map to comparators, table lookups, and short history buffers. Third, the evaluation should separate persistence-heavy easy cases from genuine transition prediction so that raw accuracy does not overstate usefulness for adaptive control.

3 Related Work

Early phase research established that large programs revisit similar behaviors over long executions. Basic Block Distribution Analysis and SimPoint use basic-block vectors to cluster intervals and select representative simulation points, showing that code-frequency fingerprints can summarize large-scale execution behavior efficiently [31, 32, 13]. Sherwood et al. later moved toward online phase tracking and prediction with compact fingerprints and run-length-aware Markov prediction, demonstrating that phase transitions are the hard cases for simple predictors [33]. Other work compared phase structures based on basic blocks, procedures, working sets, and compact signatures, making clear that a phase definition is not universal: it should be chosen for the control or analysis task it supports [9, 10, 21].

Parallel and multicore phase work extends this idea beyond single-threaded behavior. Perelman et al. analyze phases in shared-memory applications and validate phase quality using metrics such as CPI and L2/L3 hit rates, showing that phase behavior changes with thread count and shared-resource interaction [29]. Ding et al. and Zhang et al. argue that phases can exist at multiple granularities and can be exploited by runtime systems, memory managers, and architecture-level policies

Algorithm 1: Resource-Family Phase Distillation

Input: $X = \{x_{r,t} \in \mathbb{R}^d\}, \quad M = \{m_r\}, \quad \mathcal{F} = \{\text{resource families}\}$

Output: $h_\theta : \{0,1,2\}^{|\mathcal{F}| \times H} \to \{0,1,2\}, \quad H = 20$

1. $(\mathcal{D}_{tr}, \mathcal{D}_{val}, \mathcal{D}_{te}) \leftarrow \mathrm{GroupSplit}(M)$
   Keep complete runs together so correlated intervals cannot leak across train, validation, and test splits.
2. $\{\mu_0, \mu_1, \mu_2\} \leftarrow k\text{-means}_3(\mathrm{norm}(X_{tr}))$ *offline teacher*
   Cluster the full normalized safe-counter vectors in training data to discover recurring pressure regions.
3. $z_t \leftarrow \arg\min_{j \in \{0,1,2\}} \|\mathrm{norm}(x_t) - \mu_j\|_2$ *nearest-centroid label*
   Assign every interval to the closest teacher centroid, including validation and test intervals.
4. $z_t \leftarrow \mathrm{OrderByPressure}(z_t) \in \{\text{Phase } 1, \text{Phase } 2, \text{Phase } 3\}$
   Rename anonymous cluster IDs by increasing resource pressure so the labels become interpretable.
5. $\mathcal{R}_f \leftarrow \mathrm{CorrPrune}(\{c : c \in f\}, X_{tr}), \qquad \forall f \in \mathcal{F}$
   Use train-set correlation analysis to remove near-duplicate counters within each resource family.
6. $c_f^\star \leftarrow \arg\max_{c \in \mathcal{R}_f} \mathrm{AblationScore}_{val}(c, z), \qquad \forall f \in \mathcal{F}$
   Run singleton counter ablations and keep the validation-best representative for each family.
7. $s_{f,t} \leftarrow C_f(c_f^\star(t)), \qquad C_f : \mathbb{R} \to \{0,1,2\}$
   Discretize each selected counter into a compact low, moderate, or high family-state signal.
8. $u_t \leftarrow [s_{\cdot,t-H+1}, \ldots, s_{\cdot,t}]$
   Build the online feature vector from the most recent $H = 20$ resource-family state intervals.
9. $h_\theta \leftarrow \mathrm{Train}\{(u_t, z_{t+1}) : t \in \mathcal{D}_{tr}\}; \quad \mathrm{Eval}(h_\theta, \mathcal{D}_{te})$
   Train the next-phase predictor on training histories and evaluate it only on held-out traces.
10. $\mathrm{Report}\{\text{accuracy}, \text{transition accuracy}, \text{latency}, \text{storage}, \text{complexity}\}$
    Report both prediction quality and hardware feasibility instead of accuracy alone.

Hardware rule: deploy h_θ over clustered selected-counter histories; keep full-counter clustering offline.

Algorithm 1: Mathematical pseudocode for the proposed resource-family phase pipeline. The full safe-counter teacher defines labels offline; correlation pruning and ablation select the compact online counter interface; the hardware-facing predictor consumes only family-state histories.

Table 1: Common phase definitions for multicore workloads and their trade-offs. Global definitions are compact but can hide per-core behavior; local definitions specialize by core but miss shared-resource effects; distributed definitions expose cross-core context but increase noise and cost.

Definition	Description	Advantages	Limitations
Global	One phase label describes the whole processor or workload group at each interval.	Simple control interface; low storage and model overhead; well matched to highly synchronized multithreaded workloads and system-wide controls such as package power limits.	Masks asynchronous thread behavior; a change in one core can force a global phase transition; aggregate labels can obscure which core or resource caused the change.
Local	Each core or thread has its own phase sequence, either with shared model parameters or with per-core models.	Captures independent per-core behavior; avoids forcing unrelated threads into one label; often stronger for workload forecasting when cores progress asynchronously.	Ignores interference unless shared-resource counters are added; can create many small per-core phases; per-core models increase storage, training, and validation cost.
Distributed	Each core has a local phase, but prediction uses that core's history together with other cores' phase or counter context.	Represents both per-core specialization and cross-core effects; can learn that similar behavior appears on different cores at different times.	Sensitive to noisy or irrelevant cross-core inputs; higher feature and model cost; more complex hardware interface for low-latency control.

[11, 35]. Sampling-based multithreaded phase classification further shows that online classifiers can avoid full instrumentation while still tracking thread-level behavior [6]. These works motivate the paper's explicit separation of single-process multithreaded execution, multiprocess interference, and hybrid multiprocess/multithread execution.

Hardware-performance-counter approaches make phase information more directly deployable. Nomani and Szefer predict phases using hardware counters and use the predictions for scheduling decisions that reduce harmful co-scheduling and side-channel exposure [28]. Ziedan et al. focus on per-phase L2 cache demand, showing that cache needs can vary by phase and can be measured at runtime [36]. Kim et al.'s P^4 framework selects performance counters statistically and predicts power/performance behavior across heterogeneous systems, with counters such as LLC references, LLC misses, branch events, and L1 data-cache activity emerging as useful signals [20]. Alcorta Lozano and Gerstlauer combine phase classification, phase prediction, and workload forecasting for multicore CPUs, evaluating global, local, and distributed definitions and using counters tied to memory boundedness and control-flow predictability [25]. Carpentieri et al. extend the phase-aware control idea to frequency scaling for heterogeneous GPU execution, reinforcing the broader point that phase labels are useful

when they connect to resource decisions such as DVFS [5].

This paper differs from those systems in both its hardware-budget emphasis and its resource-family phase definition. Rather than assigning phases primarily to cores, threads, or whole workloads, it defines the deployed phase interface around hardware resource families. The full-counter clustering model remains an offline teacher, while the online artifact freezes one representative counter per resource family, discretizes those counters into family-specific phase states, and evaluates shallow predictors against larger software baselines. The goal is not to replace richer phase-analysis methods, but to distill their useful structure into a small set of resource-aligned signals that could plausibly feed power, thermal, cache, prefetch, or scheduling control loops.

4 Methodology

Algorithm 1 gives the full algorithmic pipeline from trace collection through held-out evaluation of a hardware-facing resource-family phase interface. The rest of this section expands each stage and explains how leakage is avoided between label construction, counter selection, and final reporting.

4.1 Datasets and Split Protocol

The evaluation uses PARSEC workloads collected at 10 ms intervals on three multicore construction regimes: (i) a single multithreaded process, (ii) two concurrent single-threaded processes, and (iii) two concurrent processes with four threads each. The completed final run uses the `simlarge` PARSEC input and covers seven workloads: blackscholes, bodytrack, canneal, fluidanimate, freqmine, swaptions, and streamcluster. Table 2 summarizes the resulting artifacts.

The split protocol is configuration-aware. Train/validation/test partitions are formed with grouped holdout so that repetitions of the same collection configuration do not cross split boundaries. The offline phase clustering teacher is fit only on training rows. Counter-family ablation uses validation rows to select representative counters. Final online predictor results are reported only on the held-out test split.

4.2 Counter Families and Safe Features

The safe counter pool excludes timing-derived features such as cycles, IPC/CPI, elapsed time, and directly frequency-sensitive rates. This prevents the predictor from learning labels that simply echo the effect of the control policy itself, which is especially important if the detector is later placed inside an adaptive hardware loop. This choice follows the usual caution around PMU interpretation: some counters are architecturally meaningful control signals, while others mainly reflect derived timing behavior or measurement context [12, 16, 17, 22]. The remaining event families are:

- L1 behavior,
- LLC behavior,
- branch/control behavior,
- core instruction/floating-point behavior,
- memory/off-core behavior.

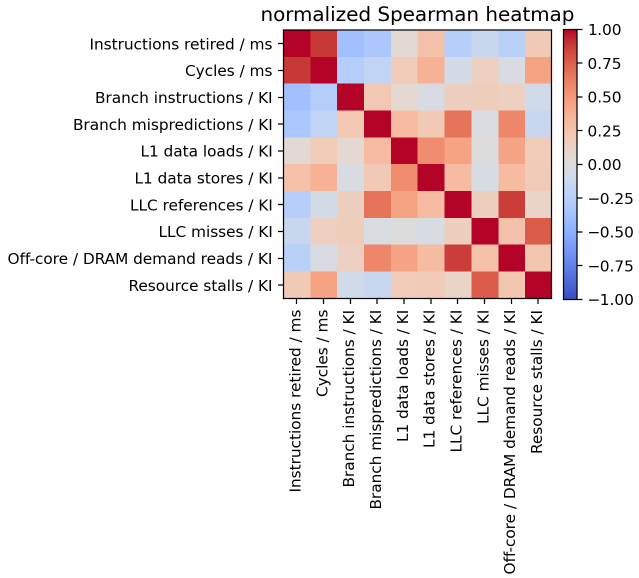


Figure 1: Normalized Spearman correlation heatmap from the counter-reduction study. The detector uses this study to avoid monitoring near-duplicate events.

The normalized correlation study confirms that several counters are strongly redundant. Figure 1 shows the normalized Spearman structure used to prune obvious near-duplicates before ablation. The final online interface therefore monitors only one representative counter per resource family, turning a broad PMU view into a compact set of resource-aligned hardware signals.

4.3 Representative Counter Selection

Each candidate counter is evaluated as a singleton resource-family representative against the offline phase labels. The selected counters for the final run are shown in Table 3. The selected memory counter varies across regimes, and Set 2 prefers branch mispredictions rather than branch volume. This reinforces a practical point for adaptable hardware: event reduction should be matched to the intended deployment regime rather than assumed universal. That lesson is consistent with earlier phase-classification comparisons, which found that no single structural view dominates every workload context [10, 21].

Table 2: Final dataset artifacts used in this paper. Set 1 sweeps thread count, while Sets 2 and 3 keep process/thread structure fixed in the completed run.

Set	Regime	Runs	Interval rows	Train rows for clustering	Configuration values	Input
Set 1	single-process multithreaded	210	457,575	326,714	threads $\in \{1, 4, 16\}$	simlarge
Set 2	two single-threaded processes	80	24,924	19,341	process count $= 2$	simlarge
Set 3	two 4-thread processes	80	127,620	101,084	threads $= 4$, process count $= 2$	simlarge

Table 3: Validation-selected representative counters for the completed final run.

Set	L1	LLC	Branch/control	Core/FP	Memory/off-core
Set 1	l1d_loads	llc_references	branch_instructions	instructions_retired	total_memory_bandwidth
Set 2	l1d_stores	llc_references	branch_mispredictions	instructions_retired	offcore_demand_data_reads
Set 3	l1d_loads	llc_misses	branch_instructions	instructions_retired	total_memory_bandwidth

4.4 Offline Phase Labels by Train-Only Clustering

The paper uses a three-state label space,

$$s_t \in \{0, 1, 2\} = \{\text{low}, \text{moderate}, \text{high}\}, \tag{1}$$

but these labels are not assigned by manual thresholds. Instead, train-only k-means over the full normalized safe counter vector defines three anonymous clusters. Those clusters are then ordered by a resource-pressure score derived from normalized LLC misses, memory bandwidth, branch mispredictions, and stalls when available. Validation and test rows are labeled by nearest-centroid assignment from the fitted training model. Using offline clustering only for label construction keeps the online predictor separate from the teacher and follows the standard role of clustering as a descriptive rather than deployment-stage tool [26, 24, 19]. For hardware, that separation is the key efficiency move: the expensive model explains the phase space offline, while the deployed mechanism only tracks the compact state that remains.

This ordering step is what makes the labels interpretable. The clustering stage is free to discover recurring behaviors in the full counter space, but the final low/moderate/high names are assigned only after checking which cluster corresponds to increasing pressure. Figure 3 visualizes the ordered centroid structure and the resulting pressure scores, while Table 4 reports the numeric summary.

The ordered pressure gaps are large enough to justify the low/moderate/high interpretation in all three sets, although the moderate state is clearly the least cleanly separated one. In Set 2 especially, the moderate cluster behaves more like a narrow mixed region between two stronger modes than a broad standalone operating point.

4.5 Online Prediction Task

The deployed hardware-facing task is single-step prediction: given a history of the last 20 clustered resource-family states, predict the next phase label. No online model receives raw counter values. Every online input is a clustered resource-family history learned from train-only one-dimensional clustering of the selected counters. This keeps the deployment interface discrete, resource aligned, and realistic for table- and comparator-based implementations.

4.6 Compared Models

The evaluation compares simple persistence baselines, compact hardware-oriented predictors, and larger software sequence models. The last-state baseline predicts that the next phase is unchanged, testing how much of the task is explained by phase persistence alone. The majority baseline always predicts the most common training phase, while the state-conditioned majority baseline predicts the most common next phase for the current state. The first-order Markov predictor uses the empirical transition matrix between phase labels, and the run-length-aware Markov variant additionally conditions on how long the current phase has already lasted. The HSMM-style approximation extends this duration idea with a lightweight state-duration model, approximating semi-Markov behavior without full probabilistic inference.

The shallow decision trees are the main hardware-oriented learned models. The single-family tree predicts from one resource family's clustered history, which tests whether a local resource signal is enough; the cross-family tree observes all selected family histories, allowing cache, branch/control, core, and memory/off-core interactions to influence the decision. ROCKET-style features provide a stronger but less hardware-friendly convolutional feature baseline by applying many random temporal filters before classification. The small TCN uses learned causal convolutions to capture short temporal patterns, and the tiny Transformer uses attention over the history window as an upper-bound sequence model. These modern sequence models serve as software baselines rather than deployment targets. They are useful for understanding the remaining headroom after counter reduction, but their arithmetic intensity and storage requirements are fundamentally different from those of the shallow-tree design and therefore less aligned with efficient hardware adaptation. Their inclusion also mirrors the progression from recurrent sequence models to temporal convolutions,

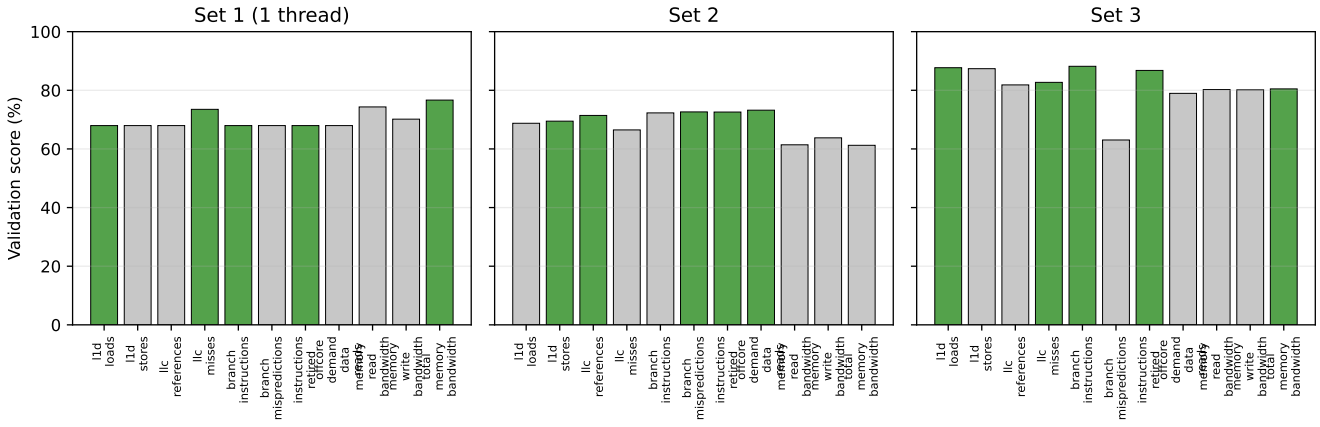


Figure 2: Singleton resource-family ablation scores on the validation split. Green bars are the selected representative counters for each family.

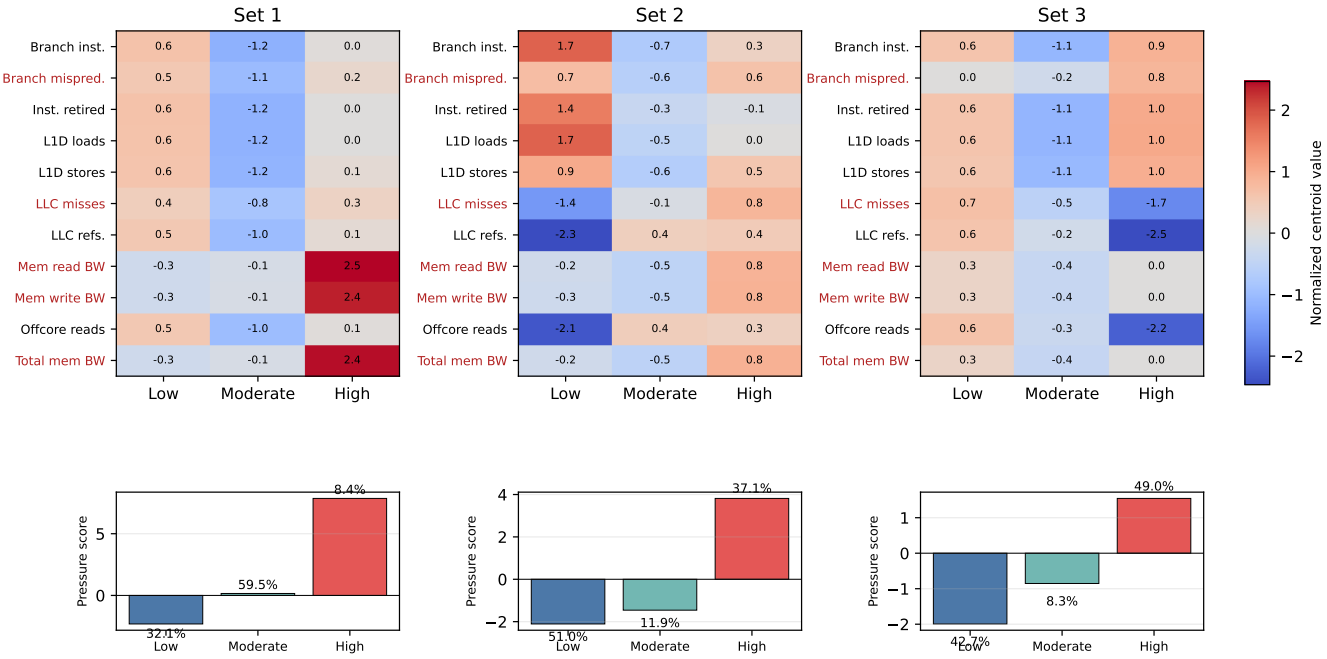


Figure 3: Interpretability of the clustered phase labels. Top: ordered cluster centroids over the full safe counter set. Pressure counters are highlighted in red. Bottom: the pressure score used to rename the clusters as low, moderate, and high, with cluster shares annotated.

random convolutional kernels, and attention-based encoders in the broader forecasting literature [15, 2, 7, 34].

4.7 Metrics and Hardware Budget

For each family and model, the paper reports accuracy, macro F1, weighted F1, balanced accuracy, high-state recall, stable-region accuracy, transition accuracy, confusion matrices, per-workload accuracy, per-core accuracy, training time, inference latency, and an approximate deployment budget. The hardware budget includes stored parameters or table entries, estimated memory in bytes, approximate operations per prediction, dominant operator type, a coarse complexity category, and a deployment recommendation.

5 Results

5.1 What the Labels Mean

The interpretability plot in Figure 3 makes the labeling story concrete. The high state consistently carries the largest positive values on LLC misses and memory bandwidth counters, while the low state sits at the opposite end of that axis. The moderate state is not just a midpoint by construction; it is the cluster

Table 4: Ordered pressure scores and cluster shares for the train-only clustered labels. Higher pressure score means more memory/cache/control pressure after normalization.

Set	Ordered pressure scores (low / moderate / high)	Cluster shares (low / moderate / high)
Set 1	$-2.31/0.16/7.86$	$32.1\%/59.5\%/8.4\%$
Set 2	$-2.11/-1.46/3.82$	$51.0\%/11.9\%/37.1\%$
Set 3	$-1.99/-0.85/1.54$	$42.7\%/8.3\%/49.0\%$

that remains after the pressure sort and usually retains mixed behavior across cache, branch, and off-core dimensions.

This matters for the adaptable-hardware claim. The online predictor is not learning arbitrary tertile buckets. It is learning to imitate a richer offline classifier whose labels already summarize multi-counter behavioral similarity, much closer in spirit to classic clustering-based phase analysis than to ad hoc thresholding [32, 13, 19]. Because those states are organized around resource families, the compact state is useful as a control signal rather than merely a compressed statistic.

5.2 Comparison Across Model Families

Figure 4 compares representative approaches on held-out test data, emphasizing the trade-off between adaptation quality and implementation cost. Several patterns are clear. First, persistence is extremely strong in Sets 2 and 3 because the test configurations contain long stable runs. This causes last-state, state-conditioned majority, and Markov prediction to collapse to nearly the same result. Second, the single-family tree is the most transition-sensitive of the low-cost models, but it pays for that aggressiveness with a substantial drop in aggregate accuracy. Third, the cross-family tree recovers most of that lost accuracy by using clustered histories from all selected resource families.

The cross-family tree reaches 69.6%, 76.0%, and 91.5% mean test accuracy in Sets 1–3. Relative to the single-family tree, it improves accuracy by 5.3, 17.2, and 11.4 percentage points, respectively, which indicates that interactions among resource families matter. Compared with the tiny Transformer upper bound, the cross-family tree is within 0.8 percentage points in Set 1, beats it by 0.8 points in Set 2, and trails it by only 0.4 points in Set 3. The tree therefore captures most of the achievable accuracy without inheriting the Transformer's cost, which is the central efficiency argument behind using it as the hardware-oriented predictor.

Table 5 explains why Set 3 looks unusually strong in both validation and test accuracy. Its selected counters align much more cleanly with the offline phase labels, and the resulting held-out phase process is far more persistent than in Sets 1 and 2. In the completed artifact, Set 3 has an 8.5% transition rate and an average run length of 35.0 intervals, compared with 31.1% and 6.2 intervals for Set 1. That makes next-phase prediction substantially easier even before a sophisticated model is applied. The near-perfect match between Set 3 last-state accuracy and cross-family tree accuracy therefore indicates that Set 3 is mainly a high-persistence regime, not that hybrid execution is intrinsically easier in general. Set 2 is the more interesting contrast case: it is still persistent enough for last-state prediction to be strong, but its 19.5% transition rate leaves more room for learned transition modeling than Set 3.

Transition behavior tells a different story. The single-family tree reaches 27.9%, 41.2%, and 30.4% transition accuracy in Sets 1–3, versus 13.1%, 45.9%, and 19.3% for the cross-family tree. ROCKET-style features remain the strongest transition detector overall at 37.8%, 45.0%, and 37.3%, but they require orders of magnitude more arithmetic and storage. This suggests a practical hardware-design trade-off: single-family trees are attractive when the controller cares most about phase-change sensitivity in one resource class, while cross-family trees are the best compromise when low hardware cost and high steady-state accuracy matter more. In other words, the study reproduces a familiar systems pattern: more expressive predictors can recover transition behavior, but the implementation budget still determines whether that extra accuracy is actionable [20, 25, 2, 7, 34].

Table 6 highlights the main architectural takeaway. Across the full model suite, the cross-family tree is the cheapest predictor that still achieves non-zero transition accuracy and competitive held-out accuracy. Its mean accuracy is only 0.1 percentage points below the tiny Transformer average while using roughly $272\times$ less memory on average.

5.3 Hardware Cost Trade-offs

Figure 5 maps each representative model to estimated memory and predictive quality, making the efficiency side of the title explicit. The cross-family tree stays in the 156–307 B range depending on set, with a low-complexity comparator-style implementation. ROCKET and TCN retain useful accuracy, but their budgets move immediately into the software-runtime regime. The tiny Transformer improves raw accuracy only marginally beyond the tree in Sets 1 and 3 and loses to the tree in Set 2, yet it requires around 69.8 KB and hundreds of microseconds of inference latency per window.

Among the purely table- and comparator-style models, last-state and Markov are cheapest but provide no transition sensitivity. The HSMM approximation adds duration awareness at low cost, but its transition gains are limited. The cross-family tree therefore occupies a useful middle ground: it is small enough for a simple hardware implementation, yet expressive enough to recover nearly all of the raw-accuracy headroom that remains after discretization. This is the point at which efficiency and adaptability meet in the design space.

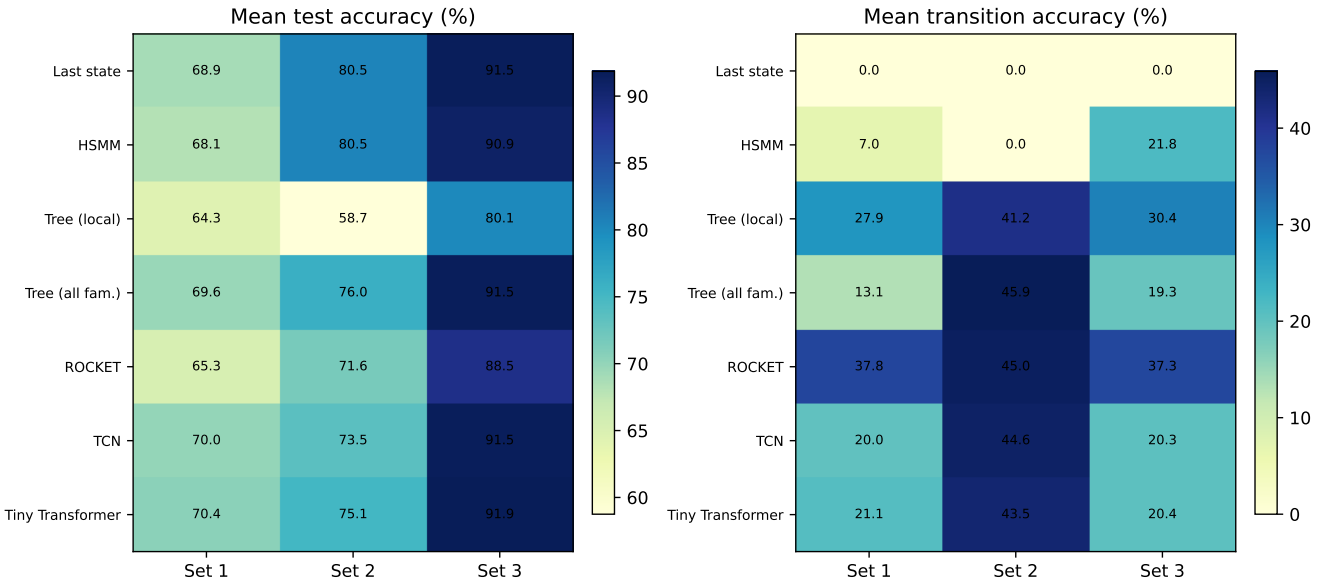


Figure 4: Held-out test comparison across representative models. The left panel shows mean accuracy; the right panel shows transition accuracy. The proposed cross-family tree is competitive with much larger software models in raw accuracy, while the single-family tree is more transition-sensitive.

Table 5: Why Set 3 is easier in the completed final run. The hybrid set should not be interpreted as a broader superset of Sets 1 and 2 here; it is a narrower fixed operating point with much longer phase runs.

Set	Mean selected-counter validation acc.	Test transition rate	Avg. run length	Last-state test acc.	Cross-family tree test acc.
Set 1	73.8	31.1	6.17	68.9	69.6
Set 2	74.5	19.5	8.54	80.5	76.0
Set 3	86.8	8.5	35.03	91.5	91.5

5.4 Effect of Thread and Process Configuration

The completed final run exposes a full thread sweep only in Set 1. Sets 2 and 3 are still useful as fixed operating points for interference and hybrid execution, but they do not contain enough process-count variation in the final artifact to support a comparable curve. Figure 6 therefore focuses on Set 1.

The shift is substantial. The transition rate increases from 7.1% at one thread to 18.1% at four threads and 34.9% at sixteen threads, while the average run length drops from 19.3 to 10.7 to 4.5 intervals. Every predictor loses accuracy as a result. For the cross-family tree, mean accuracy falls from 92.9% at one thread to 83.0% at four threads and 65.9% at sixteen threads. The same monotonic decline appears in the last-state baseline and in the tiny Transformer upper bound. This is a useful sanity check for adaptive hardware: the detector is not failing randomly; it becomes harder because the underlying phase process becomes less persistent. This is also the right way to interpret Set 3 versus Set 1: the hybrid setting wins on raw accuracy here largely because its test traces sit closer to the left side of Figure 6, with long stable runs and infrequent label changes, not because hybrid workloads are inherently simpler.

5.5 What the Baselines Tell Us

Three additional observations are worth calling out. First, the state-conditioned majority baseline and the first-order Markov predictor are effectively identical in these artifacts. That indicates very strong diagonal transition structure: when only the current phase is known, the most likely next state is usually to remain where the process already is. Second, the single-family tree and ROCKET models achieve the strongest transition accuracies, which suggests that more reactive models can be useful when action changes are cheap. Third, the cross-family tree, TCN, and tiny Transformer form a remarkably tight accuracy band despite their very different budgets. This is a strong sign that the clustered-state representation has already captured much of the useful predictive structure while remaining small enough to be plausible as a hardware interface.

6 Discussion

The paper's main claim is not that shallow trees are universally the most accurate models. They are not. The more precise claim is that train-only clustered phase labels, resource-family counter reduction, and cross-family clustered-state histories allow a shallow tree to retain most of the useful predictive power

Table 6: Representative model comparison averaged over Sets 1–3. The cross-family tree is the main proposed predictor because it provides the strongest balance between adaptation signal and hardware efficiency.

Model	Acc.	Macro F1	Bal. acc.	Trans. acc.	Memory (B)	Deployment
Last state	80.3	78.7	78.6	0.0	1	simple hardware
HSMM approximation	79.8	75.6	74.1	9.6	78	simple hardware
Tree (single family)	67.7	53.8	56.9	33.2	203	simple hardware
Tree (all families)	79.0	76.9	75.9	26.1	257	simple hardware
ROCKET (all families)	75.1	66.5	65.0	40.0	8,192	firmware/software runtime
TCN (all families)	78.3	75.6	74.0	28.3	4,748	firmware/software runtime
Tiny Transformer (all families)	79.1	77.4	77.0	28.3	69,772	upper bound only

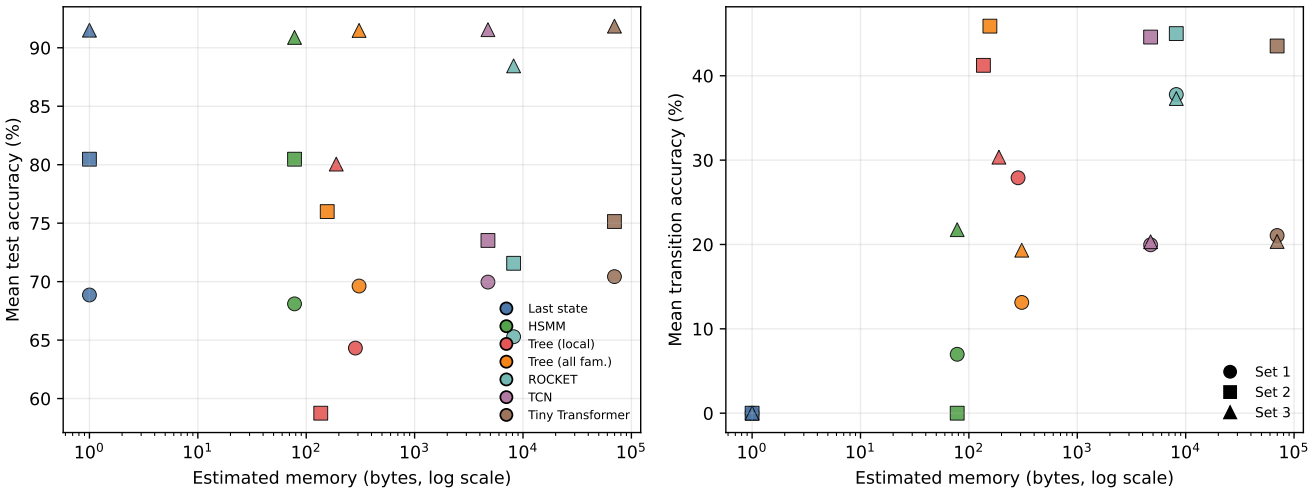


Figure 5: Accuracy and transition trade-offs versus estimated model memory. Colors denote model class and marker shapes denote sets. The cross-family tree stays close to the lower-left hardware-feasible corner, while ROCKET/TCN/Transformer models consume far more memory for modest gains.

of much larger models at far lower cost. That is why the work is best read as a step toward efficient and adaptable hardware rather than only as another phase-classification benchmark. That claim is supported by the final run.

The results also sharpen when different models should be preferred in an adaptive multicore design. If a deployment target values raw accuracy on stable workloads above all else, last-state persistence is already strong. If the target values transition sensitivity and can afford more software cost, ROCKET-like features are attractive. If the target requires a low-complexity resource-family interface with modest but non-zero transition awareness, the cross-family tree is the best compromise in this study.

6.1 Pilot L1 Control-Policy Simulation

As a first step toward closing the loop, the artifact now includes a small trace-level L1 policy driven by the predicted next L1 phase. The goal is deliberately narrow: given the phase predictor already evaluated in the paper, ask whether its output can be converted into a plausible hardware action and whether that action creates a visible power-saving opportunity. For this pilot, only the L1 family is controlled. The policy maps a predicted low L1 phase to a conservative low-power cache mode, a moderate phase to a lightly reduced mode, and a high phase to the full L1 configuration. In the modeled policy, these modes use relative L1 power factors

$$P_{\mathrm{L1}}(0) = 0.82, \qquad P_{\mathrm{L1}}(1) = 0.94, \qquad P_{\mathrm{L1}}(2) = 1.00, \tag{2}$$

where state 0 is low pressure, state 1 is moderate pressure, and state 2 is high pressure. The absolute values are not claimed as a calibrated circuit model; they are a simple normalized proxy for reducing voltage, active ways, or an equivalent L1 operating mode. A two-interval confirmation filter is applied before changing modes so that one noisy prediction does not immediately reconfigure the cache.

Figure 7 reports three quantities. The guarded-policy bar is the modeled saving relative to an always-full L1 mode:

$$\text{Saving} = 1 - \frac{1}{T}\sum_{t=1}^{T} P_{\mathrm{L1}}(a_t), \tag{3}$$

where a_t is the filtered action selected from the predicted next phase. The oracle bar uses the true next phase in place of the predicted phase, so it is an upper-bound reference rather than a deployable policy. The underprovision bar counts intervals

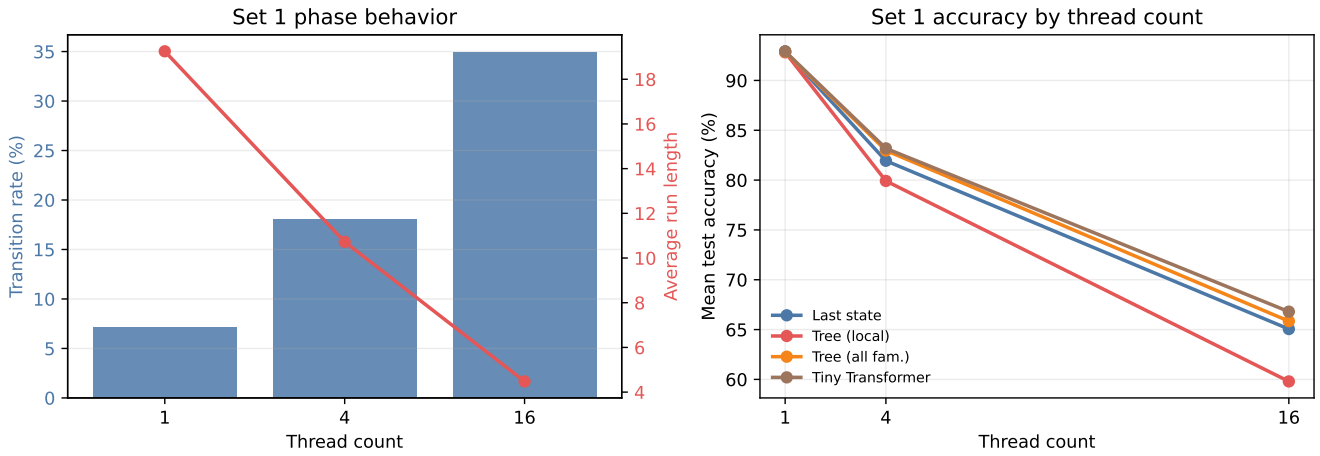


Figure 6: Set 1 thread scaling. Left: the phase process becomes less persistent as thread count grows. Right: detector accuracy falls for all models as transitions become more frequent.

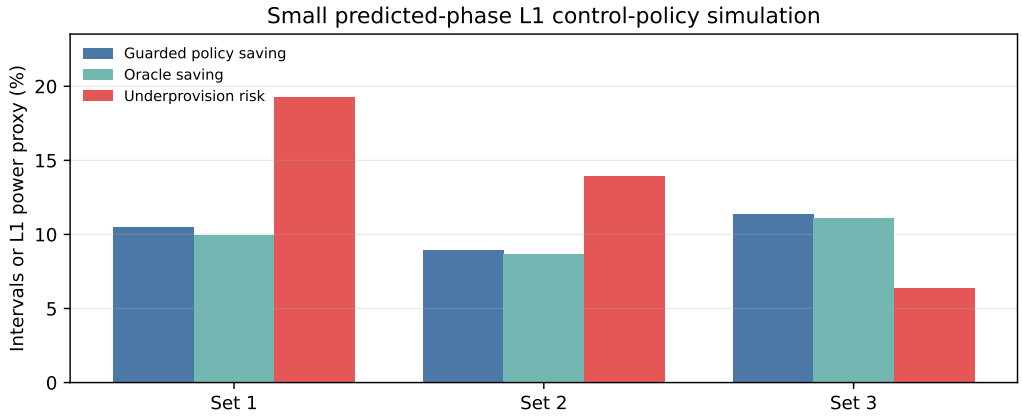


Figure 7: Pilot L1 control-policy simulation using predicted one-step L1 phases from the cross-family tree. Blue bars show modeled L1 power savings after the two-interval confirmation filter. Teal bars show an oracle upper bound that uses the true next L1 phase. Red bars show underprovision risk, where the selected action is smaller than the true next L1 phase demand.

where the selected action is below the true next L1 demand, which is the main performance-risk proxy in this simplified analysis.

The pilot result is encouraging but intentionally modest. The guarded policy estimates 10.5%, 9.0%, and 11.4% relative L1 power-proxy savings for Sets 1–3, respectively. The corresponding underprovision risks are 19.3%, 14.0%, and 6.4%. Set 3 is the cleanest case: its phase predictor is most accurate and its phase runs are longest, so the policy obtains the largest modeled saving with the lowest underprovision rate. Set 1 shows the opposite trade-off. It still exposes saving opportunity, but the higher transition rate makes the control decision more fragile, which is why its risk bar is much larger.

The comparison to the oracle bars should be read carefully. In some sets the guarded policy slightly exceeds the oracle saving because an imperfect predictor can choose a smaller action than the true phase would allow. That extra saving is not automatically good; it appears together with the red underprovision bar. Thus the pilot does not claim end-to-end energy savings, because the experiment does not run a cache simulator, does not model miss-latency feedback, and does not account for the electrical cost of changing voltage or active ways. Its purpose is narrower: it demonstrates that the proposed resource-family phase representation can be converted into a concrete control signal and that the next experimental question is now well-defined, namely how much saving remains after the action policy is evaluated with real cache-performance feedback.

7 Conclusion

This work presents a cluster-distilled resource-family phase-prediction flow as a building block for efficient and adaptable hardware for multicore workloads. Offline clustering over the full safe counter set defines interpretable low-, moderate-, and high-pressure labels. Validation-set ablation reduces the online

interface to one counter per resource family. Train-only one-dimensional clustering converts those counters into discrete resource-family state histories, and a shallow cross-family tree predicts the next phase from a 20-interval window.

The final run shows that this design is both meaningful and practical for the title's hardware goal. The cluster labels line up with increasing pressure, the cross-family tree closely tracks much larger software models in raw accuracy, and the resulting hardware budget remains in the few-hundred-byte range per family predictor rather than the multi-kilobyte or tens-of-kilobyte range of software-oriented baselines. At the same time, the thread-scaling study makes clear that concurrency changes the phase process itself: more threads create shorter runs and more frequent transitions, which is exactly the regime where history-aware predictors become valuable.

The next step is to connect the detector to a real control policy. That would let the same resource-family state representation drive DVFS, cache, prefetch, or scheduling decisions and would turn the present detection study into a full systems evaluation of adaptable multicore hardware.

References

[1] David H. Albonesi. Selective cache ways: On-demand cache resource allocation. In *Proceedings of the 32nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 248–259, 1999.

[2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

[3] Rajeev Balasubramonian, David H. Albonesi, Alper Buyuktosunoglu, and Sandhya Dwarkadas. Memory hierarchy reconfiguration for energy and performance in general-purpose processor architectures. In *Proceedings of the 33rd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 245–257, 2000.

[4] Alper Buyuktosunoglu, Stanley Schuster, David Brooks, Pradip Bose, Peter Cook, and David Albonesi. An adaptive issue queue for reduced power at high performance. In *Power-Aware Computer Systems*, pages 25–39, 2001.

[5] Lorenzo Carpentieri, Antonio De Caro, Majid Salimibeni, Kaijie Fan, and Biagio Cosenza. Phase-based frequency scaling for energy-efficient heterogeneous computing. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, pages 824–836, 2025.

[6] Chih-Hao Chang, Pangfeng Liu, and Jan-Jan Wu. Sampling-based phase classification and prediction for multi-threaded program execution on multi-core architectures. In *Proceedings of the 42nd International Conference on Parallel Processing*, pages 349–358, 2013.

[7] Angus Dempster, François Petitjean, and Geoffrey I. Webb. Rocket: Exceptionally fast and accurate time series classification using random convolutional kernels. *Data Mining and Knowledge Discovery*, 34(5):1454–1495, 2020.

[8] Gaurav Dhiman, Giovanni Marchetti, and Tajana Rosing. vgreen: A system for energy-efficient computing in virtualized environments. In *Proceedings of the International Symposium on Low Power Electronics and Design*, pages 243–248, 2009.

[9] Ashutosh S. Dhodapkar and James E. Smith. Managing multi-configuration hardware via dynamic working set analysis. In *Proceedings of the 29th International Symposium on Computer Architecture*, pages 233–244, 2002.

[10] Ashutosh S. Dhodapkar and James E. Smith. Comparing program phase detection techniques. In *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 217–227, 2003.

[11] Chen Ding, Sandhya Dwarkadas, Michael C. Huang, Kai Shen, and John B. Carter. Program phase detection and exploitation. In *Proceedings of the 20th IEEE International Parallel and Distributed Processing Symposium*, 2006.

[12] Stijn Eyerman, Lieven Eeckhout, Tejas Karkhanis, and James E. Smith. A performance counter architecture for computing accurate cpi components. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 175–184, 2006.

[13] Greg Hamerly, Erez Perelman, Jeremy Lau, and Brad Calder. Simpoint 3.0: Faster and more flexible program phase analysis. *Journal of Instruction-Level Parallelism*, 7:1–28, 2005.

[14] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition, 2019.

[15] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[16] Intel Corporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 2024.

[17] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*, 2026.

[18] Canturk Isci and Margaret Martonosi. Runtime power monitoring in high-end processors: Methodology and empirical data. In *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 93–104, 2003.

[19] Anil K. Jain, M. Narasimha Murty, and Patrick J. Flynn. Data clustering: A review. *ACM Computing Surveys*, 31(3):264–323, 1999.

[20] Yeseong Kim, Pietro Mercati, Ankit More, Emily Shriver, and Tajana Rosing. p^4: Phase-based power/performance prediction of heterogeneous systems via neural networks. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, pages 683–690, 2017.

[21] Jeremy Lau, Stefan Schoenmackers, Timothy Sherwood, and Brad Calder. Structures for phase classification. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software*, 2004.

[22] Linux Kernel Developers. *perf_event_open(2) Linux Programmer's Manual*, 2024.

[23] Linux perf developers. perf: Linux profiling with performance counters. `https://perfwiki.github.io/main/`, 2024.

[24] Stuart P. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[25] Erika Susana Alcorta Lozano and Andreas Gerstlauer. Learning-based phase-aware multi-core cpu workload forecasting. *ACM Transactions on Design Automation of Electronic Systems*, 28(2), 2022.

[26] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, pages 281–297, 1967.

[27] Andreas Merkel and Frank Bellosa. Balancing power consumption in multiprocessor systems. In *Proceedings of the 1st ACM SIGOPS/EuroSys European Conference on Computer Systems*, pages 403–414, 2006.

[28] Junaid Nomani and Jakub Szefer. Predicting program phases and defending against side-channel attacks using hardware performance counters. In *Proceedings of the Fourth Workshop on Hardware and Architectural Support for Security and Privacy*, 2015.

[29] Erez Perelman, Marzia Polito, Jean-Yves Bouguet, John Sampson, Brad Calder, and Carole Dulong. Detecting phases in parallel applications on shared memory architectures. In *Proceedings of the 20th IEEE International Parallel and Distributed Processing Symposium*, 2006.

[30] Vinicius Petrucci, Orlando Loques, and Daniel Mosse. Lucky scheduling for energy-efficient heterogeneous multi-core systems. In *Proceedings of the USENIX Workshop on Hot Topics in Power-Aware Computing and Systems*, 2012.

[31] Timothy Sherwood, Erez Perelman, and Brad Calder. Basic block distribution analysis to find periodic behavior and simulation points in applications. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, pages 3–14, 2001.

[32] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. Automatically characterizing large scale program behavior. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 45–57, 2002.

[33] Timothy Sherwood, Suleyman Sair, and Brad Calder. Phase tracking and prediction. In *Proceedings of the 30th International Symposium on Computer Architecture*, pages 336–349, 2003.

[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[35] Weihua Zhang, Jiaxin Li, Yi Li, and Haibo Chen. Multilevel phase analysis. *ACM Transactions on Embedded Computing Systems*, 14(2):31:1–31:29, 2015.

[36] Ibrahim E. Ziedan, Hazem I. Shehata, and Shaymaa M. Serag. A run-time program phase detection technique for optimizing per-phase l2 cache demand. *The Egyptian International Journal of Engineering Sciences and Technology*, 20:1–9, 2016.